```
# it is preprocessing technique and this is raw data convert use important info.
```

```
# normalization is feature engg technique that tranaform data so that it has mean=0 and statrard derivation=1
```

```
import numpy as np
import pandas as pd
```

```
df=pd.read_csv("/content/credit_scoring - credit_scoring.csv")
```

```
df.head(2)
```

	Age	Gender	Marital Status	Education Level	Employment Status	Credit Utilization Ratio	Payment History	Number of Credit Accounts	Loan Amount	Interest Rate	Loan Term	Type of Loan
0	60	Male	Married	Master	Employed	0.22	2685	2	4675000	2.65	48	Personal Loan

```
df['Employment Status'].value_counts()
```

Employment Status	count
Self-Employed	347
Employed	328
Unemployed	325

dtype: int64

```
df.isnull().sum()
```

	0
Age	0
Gender	0
Marital Status	0
Education Level	0
Employment Status	0
Credit Utilization Ratio	0
Payment History	0
Number of Credit Accounts	0
Loan Amount	0
Interest Rate	0
Loan Term	0
Type of Loan	0

dtype: int64

```
from sklearn.preprocessing import LabelEncoder
lb=LabelEncoder()
```

level encodind

```
df['Gender']=lb.fit_transform(df['Gender'])
df['Marital Status']=lb.fit_transform(df['Marital Status'])
df['Employment Status']=lb.fit_transform(df['Employment Status'])
df['Education Level']=lb.fit_transform(df['Education Level'])
```

```
x=df.drop(columns=['Type of Loan'])
y=df['Type of Loan']
```

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
```

apply standardization

get dummies

```
a=pd.get_dummies(df[['Employment Status','Education Level','Gender','Employment Status']],drop_first=True)
```

```
a=a.astype(int)
```

```
a
```

	Employment Status	Education Level	Gender	Employment Status
0	0	2	1	0
1	2	1	1	2
2	0	2	0	0
3	2	3	0	2
4	1	0	1	1
...	...	...	...	...
995	0	1	1	0
996	2	0	1	2
997	1	2	0	1
998	1	3	0	1
999	1	2	0	1

1000 rows × 4 columns

onehotencoder

```
from sklearn.preprocessing import OneHotEncoder
ohe=OneHotEncoder(drop='first',sparse_output=False,dtype=np.int32)
```

```
x=ohe.fit_transform(x_train[['Gender','Education Level','Employment Status']])
```

```
df=pd.DataFrame(o,columns=['Gender','Education Level','Employment Status'])
```

```
x_train
```

```
array([[0, 0, 0, 1, 0, 1],
       [1, 0, 0, 0, 0, 1],
       [1, 0, 0, 0, 1, 0],
       ...,
       [0, 0, 1, 0, 0, 1],
       [1, 1, 0, 0, 1, 0],
       [1, 0, 0, 0, 1, 0]], dtype=int32)
```

ordinalencoder

```
from sklearn.preprocessing import OrdinalEncoder
```

```
oe=OrdinalEncoder(categories=[
             ['Male','Female'],
             ['Married','Divorced','Single'],
             ['Master','High School','PhD','Bachelor'],
             ['Self-Employed','Employed','Unemployed']
         ]),
```

```
df=pd.DataFrame(df,columns=['Gender','Marital Status','Education Level','Employment Status'])
```

```
df
```

	Gender	Marital Status	Education Level	Employment Status
0	1	1	2	0
1	1	1	1	2
2	0	2	2	0
3	0	1	3	2
4	1	1	0	1
...	...	...	...	...
995	1	0	1	0
996	1	0	0	2
997	0	2	2	1
998	0	1	3	1
999	0	1	2	1

1000 rows × 4 columns

Double-click (or enter) to edit

```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
```

```
np.round(x_train.describe(),2)
```

	Age	Gender	Marital Status	Education Level	Employment Status	Credit Utilization Ratio	Payment History	Number of Credit Accounts	Loan Amount	Interest Rate	Loan Term
count	800.00	800.0	800.00	800.00	800.00	800.00	800.00	800.00	800.00	800.00	800.00
mean	42.83	0.5	1.00	1.51	1.01	0.51	1476.59	5.62	2516100.00	10.58	37.48
std	13.23	0.5	0.81	1.11	0.81	0.29	819.51	2.94	1388854.42	5.44	17.32
min	20.00	0.0	0.00	0.00	0.00	0.00	0.00	1.00	108000.00	1.01	12.00
25%	31.75	0.0	0.00	1.00	0.00	0.24	800.00	3.00	1308750.00	5.99	24.00
50%	42.00	1.0	1.00	2.00	1.00	0.53	1457.00	6.00	2497000.00	10.47	36.00
75%	54.25	1.0	2.00	2.00	2.00	0.77	2171.00	8.00	3699000.00	15.33	48.00

```
sc=sc.fit_transform(x_train)
```

```
sc_new=pd.DataFrame(sc,columns=x_train.columns)
```

```
np.round(sc_new.describe(),2)
```

	Age	Gender	Marital Status	Education Level	Employment Status	Credit Utilization Ratio	Payment History	Number of Credit Accounts	Loan Amount	Interest Rate	Loan Term
count	800.00	800.00	800.00	800.00	800.00	800.00	800.00	800.00	800.00	800.00	800.00
mean	-0.00	0.00	-0.00	0.00	-0.00	0.00	-0.00	-0.00	0.00	0.00	0.00
std	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
min	-1.73	-1.01	-1.23	-1.36	-1.25	-1.75	-1.80	-1.57	-1.73	-1.76	-1.47
25%	-0.84	-1.01	-1.23	-0.46	-1.25	-0.93	-0.83	-0.89	-0.87	-0.84	-0.78
50%	-0.06	1.00	0.01	0.44	-0.01	0.06	-0.02	0.13	-0.01	-0.02	-0.09
75%	0.86	1.00	1.24	0.44	1.23	0.88	0.85	0.81	0.85	0.87	0.61

apply normalization

```
from sklearn.preprocessing import MinMaxScaler
mn=MinMaxScaler()
```

```
np.round(x_train.describe(),2)
```

	Age	Gender	Marital Status	Education Level	Employment Status	Credit Utilization Ratio	Payment History	Number of Credit Accounts	Loan Amount	Interest Rate	Loan Term
count	800.00	800.0	800.00	800.00	800.00	800.00	800.00	800.00	800.00	800.00	800.00
mean	42.83	0.5	1.00	1.51	1.01	0.51	1476.59	5.62	2516100.00	10.58	37.48
std	13.23	0.5	0.81	1.11	0.81	0.29	819.51	2.94	1388854.42	5.44	17.32
min	20.00	0.0	0.00	0.00	0.00	0.00	0.00	1.00	108000.00	1.01	12.00
25%	31.75	0.0	0.00	1.00	0.00	0.24	800.00	3.00	1308750.00	5.99	24.00
50%	42.00	1.0	1.00	2.00	1.00	0.53	1457.00	6.00	2497000.00	10.47	36.00
75%	54.25	1.0	2.00	2.00	2.00	0.77	2171.00	8.00	3699000.00	15.33	48.00

```
mn=mn.fit_transform(x_train)
```

```
mn_new=pd.DataFrame(mn,columns=x_train.columns)
```

```
np.round(mn_new.describe(),2 )
```

	Age	Gender	Marital Status	Education Level	Employment Status	Credit Utilization Ratio	Payment History	Number of Credit Accounts	Loan Amount	Interest Rate	Loan Term
count	800.00	800.0	800.0	800.00	800.0	800.00	800.00	800.00	800.00	800.00	800.00
mean	0.51	0.5	0.5	0.50	0.5	0.51	0.52	0.51	0.49	0.50	0.53
std	0.29	0.5	0.4	0.37	0.4	0.29	0.29	0.33	0.28	0.29	0.36
min	0.00	0.0	0.0	0.00	0.0	0.00	0.00	0.00	0.00	0.00	0.00
25%	0.26	0.0	0.0	0.33	0.0	0.24	0.28	0.22	0.25	0.26	0.25
50%	0.49	1.0	0.5	0.67	0.5	0.53	0.51	0.56	0.49	0.50	0.50
75%	0.76	1.0	1.0	0.67	1.0	0.77	0.76	0.78	0.73	0.76	0.75

pipeline

columntransformer

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OrdinalEncoder

transformer = ColumnTransformer(
    transformers=[
        ('tnf1', OrdinalEncoder(categories=[['Male','Female'],  ['Married','Divorced','Single'],['Master','High School','PH
        ['Self-Employed','Employed','Unemployed']]), ['Gender','Marital Status','Education Level','Employment Status'])
    ],
    remainder='passthrough'
)
```

```
transformer.fit_transform(x_train).shape
```

```
(800, 11)
```

```
transformer.fit_transform(x_test).shape
```

```
(200, 11)
```

pipeline

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import OrdinalEncoder
from sklearn.pipeline import Pipeline
```

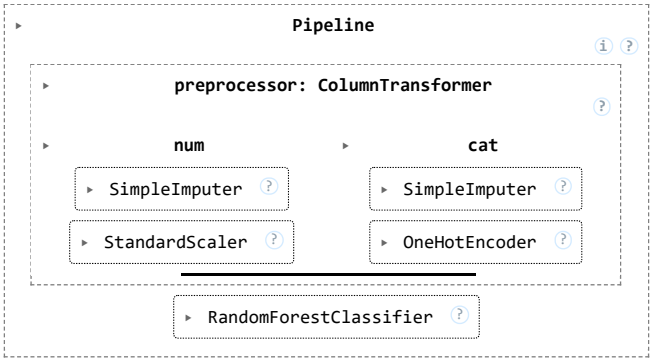
```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
categorical_features=['Gender','Marital Status']
numerical_features=['Age','Loan Amount']
```

```
numerical_transformer=Pipeline(steps=[
    ('imputer',SimpleImputer(strategy='mean')),
    ('scaler',StandardScaler())
])

categorical_transformer=Pipeline(steps=[
    ('imputer',SimpleImputer(strategy='most_frequent')),
    ('onehot',OneHotEncoder(handle_unknown='ignore'))
])
```

```
preprocessor=ColumnTransformer(transformers=[
    ('num',numerical_transformer,numerical_features),
    ('cat',categorical_transformer,categorical_features)
])
```

```
rf.fit(x_train,y_train)
```




```
y_pred=rf.predict(x_test)
```

```
accuracy_score(y_test,y_pred)
```

```
0.3
```

```
df=pd.read_csv("/content/newplacementdata - newplacementdata.1.csv")
```

```
df.head(2)
```

	cgpa	placement_exam_marks	placed
0	7.19	26	1
1	7.46	38	1

```
import matplotlib.pyplot as plt
import seaborn as sns
```

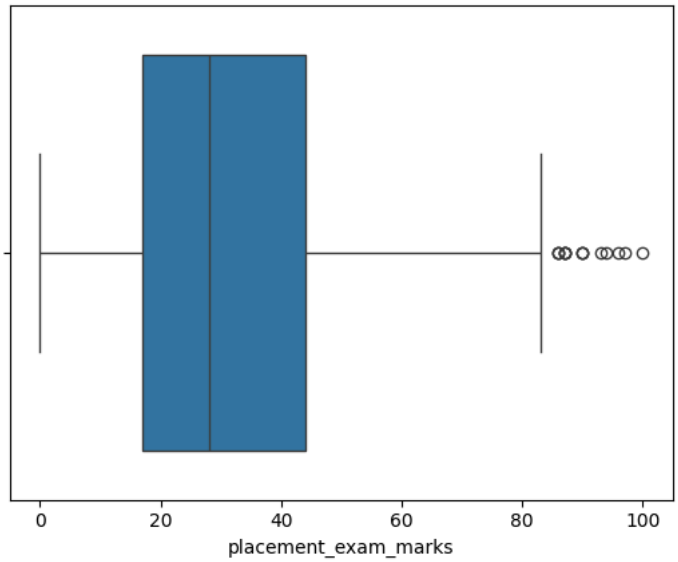
```
df['placement_exam_marks'].describe()
```

	placement_exam_marks
count	1000.000000
mean	32.225000
std	19.130822
min	0.000000
25%	17.000000
50%	28.000000
75%	44.000000
max	100.000000

dtype: float64

```
sns.boxplot(x=df['placement_exam_marks'])
```

```
<Axes: xlabel='placement_exam_marks'>
```




```
percentile25 = df['placement_exam_marks'].quantile(0.25)
percentile75 = df['placement_exam_marks'].quantile(0.75)
percentile25
```

```
np.float64(17.0)
```

```
percentile75
```

```
np.float64(44.0)
```

```
percentile25
```

```
np.float64(17.0)
```

```
IQR=percentile75-percentile25
IQR
```

```
np.float64(27.0)
```

```
upper_limit=percentile75+1.5*IQR
upper_limit
```

```
np.float64(84.5)
```

```
lower_limit=percentile25-1.5*IQR
lower_limit
```

```
np.float64(-23.5)
```

```
df[(df['placement_exam_marks']>upper_limit)]
```

	cgpa	placement_exam_marks	placed
9	7.75	94	1
40	6.60	86	1
61	7.51	86	0
134	6.33	93	0
162	7.80	90	0
283	7.09	87	0
290	8.38	87	0
311	6.97	87	1
324	6.64	90	0
630	6.56	96	1
685	6.05	87	1
730	6.14	90	1
771	7.31	86	1
846	6.99	97	0
917	5.95	100	0

```
df[(df['placement_exam_marks']<lower_limit)]
```

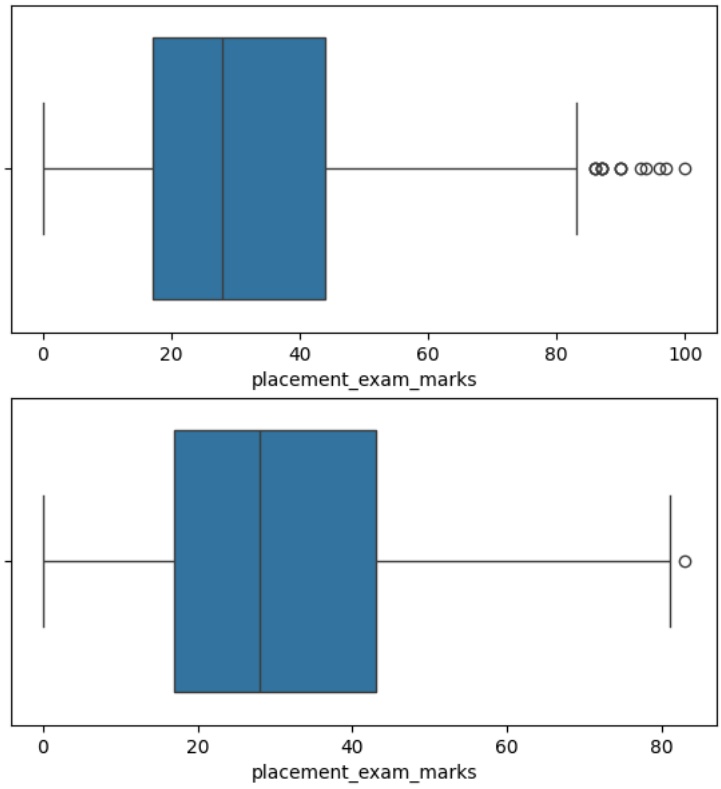
cgpa	placement_exam_marks	placed

```
newdf=df[(df['placement_exam_marks']<upper_limit)]
newdf
```

	cgpa	placement_exam_marks	placed
0	7.19	26	1
1	7.46	38	1
2	7.54	40	1
3	6.42	8	1
4	7.23	17	0
...	...	...	...
995	8.87	44	1
996	9.12	65	1
997	4.89	34	0
998	8.62	46	1
999	4.90	10	1

985 rows × 3 columns

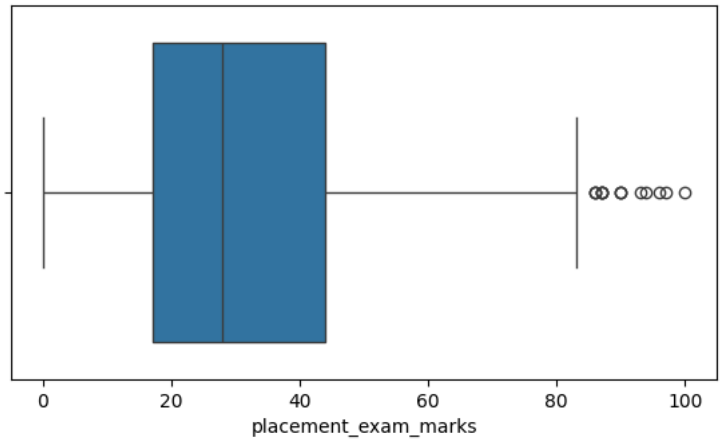
```
plt.figure(figsize=(15,7))

plt.subplot(222)
sns.boxplot(x=df['placement_exam_marks'])

plt.subplot(224)
sns.boxplot(x=newdf['placement_exam_marks'])
plt.show()
```

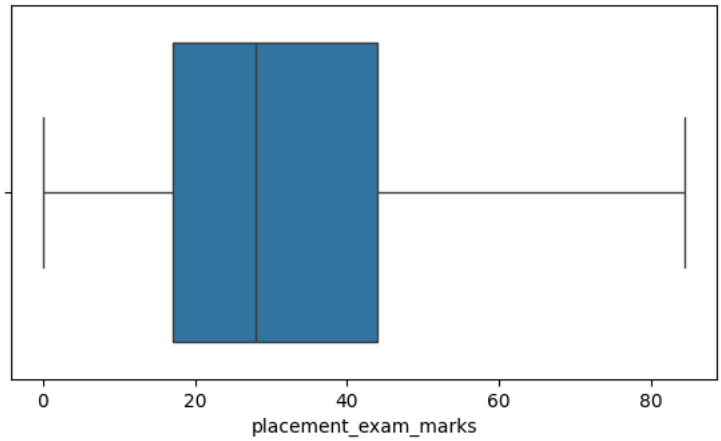



```
new_df_cap=df.copy()
```

```
new_df_cap['placement_exam_marks']=np.where(

new_df_cap['placement_exam_marks']>upper_limit,
upper_limit,

np.where(
    new_df_cap['placement_exam_marks']<lower_limit,
    lower_limit,

new_df_cap['placement_exam_marks']
)
)
```

```
plt.figure(figsize=(15,8))

plt.subplot(222)
sns.boxplot(x=df['placement_exam_marks'])

plt.subplot(224)
sns.boxplot(x=new_df_cap['placement_exam_marks'])
plt.show()
```





cca

```
df=pd.read_csv("/content/dsjob - dsjob1.csv")
```

```
df
```

	enrollee_id	city	city_development_index	gender	relevent_experience	enrolled_university	education_level	major
0	32403	city_41	0.827	Male	Has relevent experience	Full time course	Graduate	
1	9858	city_103	0.920	Female	Has relevent experience	no_enrollment	Graduate	
2	31806	city_21	0.624	Male	No relevent experience	no_enrollment	High School	
3	27385	city_13	0.827	Male	Has relevent experience	no_enrollment	Masters	
4	27724	city_103	0.920	Male	Has relevent experience	no_enrollment	Graduate	
...	...	...	...	...	...	...	...	
2124	1289	city_103	0.920	Male	No relevent experience	no_enrollment	Graduate	
2125	195	city_136	0.897	Male	Has relevent experience	no_enrollment	Masters	
2126	31762	city_100	0.887	Male	No relevent experience	no_enrollment	Primary School	
2127	7873	city_102	0.804	Male	Has relevent experience	Full time course	High School	
2128	12215	city_102	0.804	Male	Has relevent experience	no_enrollment	Masters	

2129 rows × 13 columns

```
df.isnull().sum()*100
```

	0
enrollee_id	0
city	0
city_development_index	0
gender	50800
relevent_experience	0
enrolled_university	3100
education_level	5200
major_discipline	31200
experience	500
company_size	62200
company_type	63400
last_new_job	4000

dtype: int64

```
cols= [var for var in df.columns if df[var].isnull().mean()< 0.25 and df[var].isnull().mean()>0]
cols
```

```
['gender',
 'enrolled_university',
 'education_level',
 'major_discipline',
 'experience',
 'last_new_job']
```

```
df[cols].sample(5)
```

	gender	enrolled_university	education_level	major_discipline	experience	last_new_job
924	Female	Full time course	Graduate	STEM	5	1
1518	NaN	no_enrollment	Graduate	STEM	19	3
1501	Male	no_enrollment	Masters	STEM	>20	>4
779	Male	Full time course	High School	NaN	2	never
1647	Male	Part time course	Graduate	STEM	5	4

```
df['education_level'].value_counts()
```

education_level	count
Graduate	1269
Masters	496
High School	222
Phd	54
Primary School	36

dtype: int64

```
len(df[cols].dropna())/ len(df)
```

```
0.6510098637858149
```

```
new_df= df[cols].dropna()
df.shape, new_df.shape
```

```
((2129, 13), (1386, 6))
```

```
import matplotlib.pyplot as plt
```

```
fig= plt.figure()
ax=fig.add_subplot(111)

# original data
df['experience'].hist(bins=50,ax=ax,density=True,color='red')
```